

ML Practical 1

```
In [1]: import pandas as pd
        from sklearn.model_selection import train_test_split
```

```
In [2]: #Load the dataset of healthcare
        df=pd.read_csv("GDM_Dataset.csv")
        df.head()
```

```
Out[2]:
```

	patient_id	age	pre_pregnancy_bmi	ethnicity	family_history_dm	previous_gdm	pcos
0	P0001	32.1	22.2	Caucasian	0	0	0
1	P0002	28.8	24.0	Other	1	0	1
2	P0003	32.9	22.6	African	0	0	1
3	P0004	37.4	26.0	African	0	0	0
4	P0005	28.3	31.2	Hispanic	1	0	1

```
In [3]: print("Dataset Shape:", df.shape)

        print("\nData Types:")
        print(df.dtypes)
```

Dataset Shape: (10000, 17)

Data Types:

patient_id	object
age	float64
pre_pregnancy_bmi	float64
ethnicity	object
family_history_dm	int64
previous_gdm	int64
pcos	int64
previous_macrosomia	int64
booking_gestational_age	float64
early_rbs_mgdl	float64
early_ppbs_mgdl	float64
early_hba1c_percent	float64
early_ogtt_performed	int64
early_ogtt_fasting_mgdl	float64
early_ogtt_1h_mgdl	float64
early_ogtt_2h_mgdl	float64
gdm_outcome	int64
dtype:	object

```
In [11]: print("Missing Values:")
          print(df.isnull().sum())
```

```

Missing Values:
patient_id          0
age                 0
pre_pregnancy_bmi   0
ethnicity            0
family_history_dm    0
previous_gdm         0
pcos                 0
previous_macrosomia  0
booking_gestational_age  0
early_rbs_mgdl       0
early_ppbs_mgdl      0
early_hba1c_percent  0
early_ogtt_performed  0
early_ogtt_fasting_mgdl 7009
early_ogtt_1h_mgdl   7009
early_ogtt_2h_mgdl   7009
gdm_outcome          0
dtype: int64

```

```

In [12]: X = df.drop(["patient_id", "gdm_outcome"], axis=1)
         y = df["gdm_outcome"]

         print("Input Features:")
         print(X.columns)

         print("\nTarget Variable:")
         print(y.name)

```

```

Input Features:
Index(['age', 'pre_pregnancy_bmi', 'ethnicity', 'family_history_dm',
      'previous_gdm', 'pcos', 'previous_macrosomia',
      'booking_gestational_age', 'early_rbs_mgdl', 'early_ppbs_mgdl',
      'early_hba1c_percent', 'early_ogtt_performed',
      'early_ogtt_fasting_mgdl', 'early_ogtt_1h_mgdl', 'early_ogtt_2h_mgdl'],
      dtype='object')

```

```

Target Variable:
gdm_outcome

```

```

In [13]: X_train, X_test, y_train, y_test = train_test_split(
         X,
         y,
         test_size=0.2,
         random_state=42
         )

         print("Training Features:", X_train.shape)
         print("Testing Features:", X_test.shape)

         print("Training Target:", y_train.shape)
         print("Testing Target:", y_test.shape)

```

```

Training Features: (8000, 15)
Testing Features: (2000, 15)
Training Target: (8000,)
Testing Target: (2000,)

```

STEP 8 : Train a Suitable Machine Learning Model Train a classification algorithm (like Logistic Regression, Decision Tree, or KNN) using the training data (X_train and y_train) to learn the relationship between clinical features and GDM risk.

STEP 9: Generate Predictions Use the trained model to predict the target variable (gdm_outcome) on the unseen testing data (X_test).

STEP 10: Evaluate the Model Assess the model's performance by comparing its predictions against the actual outcomes (y_test) using metrics such as Accuracy, Precision, Recall, F1-score, and a Confusion Matrix.

Conclusion: The Machine Learning workflow was successfully implemented using Python and Scikit-learn. The GDM dataset of 10,000 records was loaded, preprocessed, and split into training (80%) and testing (20%) sets, successfully preparing the data for classification model training and evaluation.

In []: